ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN

Môn học: Cơ sở trí tuệ nhân tạo
Mã học phần: CSC14003

Project 1: BLOXORZ SOLVER

Sinh viên thực hiện:

25122030 – Nguyễn Hoàng Nam
25122035 – Hoàng Trung Phong
25122036 – Huỳnh Văn Phú
25122039 – Hà Chí Tâm
25122043 – Nguyễn Như Thuật

Giảng viên hướng dẫn:

GS. TS. Lê Hoài Bắc

TP. Hồ Chí Minh, Ngày 1 tháng 11 năm 2026

Mục lục

1 Tổng quan đồ án **1**
1.1 Giới thiệu bài toán Bloxorz Solver 1
1.2 Mục tiêu đồ án 1
1.3 Kế hoạch triển khai dự án 2
1.4 Phân công nhiệm vụ & Kế hoạch chi tiết theo giai đoạn 2
1.5 Bảng tỷ lệ đóng góp công việc 5

2 Kiến trúc luồng trò chơi, chức năng các thành phần hệ thống và hướng dẫn sử dụng **5**
2.1 Quy trình vận hành trò chơi 5
2.1.1 Luồng tương tác và chuyển trạng thái hệ thống 5
2.1.2 Khởi tạo hệ thống 5
2.1.3 Luồng tương tác của người chơi 5
2.1.4 Tiến trình nạp màn (Load Stage) 6
2.1.5 Cơ chế thực thi chế độ chơi & Xem phát lại lời giải 7
2.2 Kiến trúc và Chức năng các Thành phần Hệ thống 8
2.3 Hướng dẫn cài đặt & chạy chương trình 9
2.3.1 Yêu cầu môi trường & Tải mã nguồn 9
2.3.2 Thiết lập môi trường ảo (Virtual Environment) 9
2.3.3 Cài đặt thư viện phụ thuộc & Khởi chạy chương trình 9

3 Biểu diễn không gian trạng thái & cơ chế trò chơi **10**
3.1 Biểu diễn trạng thái 10
3.2 Không gian Hành động & bảng chuyển trạng thái 10
3.3 Kiểm tra nước đi Hợp lệ & điều kiện Thắng 11
3.4 Logic tương tác các loại ô đặc biệt 11
3.4.1 Ô giòn (Fragile Tiles) 11
3.4.2 Công tắc (Switches: Soft Switch, Heavy Switch, Split Switch) 11
3.4.3 Cầu nối (Bridges) 12

4 Các thuật toán sử dụng **12**
4.1 Thuật toán Breadth-First Search (BFS) 12
4.2 Thuật toán Depth-First Search, Iterative Deepening Search 12
4.2.1 Nguyên lý hoạt động & Giới hạn độ sâu 12
4.2.2 Cơ chế lưu tập trạng thái đã duyệt (Visited Set / Hashing state key) . . 12
4.3 Thuật toán Uniform Cost Search 12
4.3.1 Cấu trúc Priority Queue 12
4.3.2 Đề xuất Thiết kế Non-uniform Cost Function 13
4.4 Thuật toán A* Search 13
4.4.1 Nguyên lý thuật toán & Cài đặt (`algorithms/astar.py`) 13
4.4.2 Đề xuất thiết kế các hàm Heuristic 13
4.4.3 Chứng minh tính Admissibility và Consistency 14

5 Thực nghiệm và đánh giá **14**
5.1 Thiết kế bộ test cases 14
5.2 Phương pháp đo lường 15
5.3 Kết quả thực nghiệm 16
5.4 Phân tích và đánh giá 17

6 Kết luận và hướng phát triển **18**
6.1 Kết quả đạt được 18
6.2 Tóm tắt hướng dẫn lựa chọn thuật toán 18
6.3 Hướng phát triển 18

Tài liệu tham khảo **20**

Phụ lục **21**

Kết quả chi tiết từng màn **21**
6.4 Sử dụng Generative AI 26

Lời mở đầu

Trí tuệ nhân tạo (AI) ngày càng khẳng định vai trò quan trọng trong việc giải quyết các bài toán phức tạp, từ tối ưu hóa tài nguyên cho đến tự động hóa quy trình quyết định. Trong số đó, các thuật toán tìm kiếm (Search Algorithms) đóng vai trò là nền tảng cốt lõi giúp tác tử thông minh định hình và tìm kiếm đường đi tối ưu trong không gian trạng thái.

Đồ án **"Bloxorz Solver"** thuộc môn học *Cơ sở trí tuệ nhân tạo* được nhóm thực hiện nhằm mục đích áp dụng các lý thuyết tìm kiếm mù (BFS, DFS, UCS) và tìm kiếm có thông tin (A*) vào một bài toán trò chơi thực tế. Thông qua việc mô hình hóa không gian trạng thái, xây dựng các hàm Heuristic tối ưu và phát triển ứng dụng trực quan, đồ án không chỉ giúp nhóm củng cố kiến thức chuyên môn mà còn rèn luyện kỹ năng lập trình, phân tích và làm việc nhóm.

Chúng em xin chân thành cảm ơn thầy **GS. TS. Lê Hoài Bắc** đã hướng dẫn và truyền đạt kiến thức quý báu trong suốt quá trình học tập và thực hiện đồ án này.

1 Tổng quan đồ án

1.1 Giới thiệu bài toán Bloxorz Solver

Bloxorz là trò chơi di chuyển một khối gạch trên bảng lưới ô vuông với các quy tắc sau:

- **Khối gạch & Định hướng (Block & Orientation):** Khối gạch có kích thước $1 \times 1 \times 2$. Tại mỗi thời điểm, khối gạch luôn ở một trong 3 trạng thái định hướng chính: Đứng (Standing) tại ô (r, c), Nằm ngang (Lying horizontally) trên hai ô (r, c) và $(r, c+1)$, hoặc Nằm dọc (Lying vertically) trên hai ô (r, c) và $(r+1, c)$.
- **Hành động & Di chuyển:** Mỗi lượt di chuyển, khối gạch có thể lăn theo 4 hướng (Up, Down, Left, Right). Cách chuyển trạng thái vị trí và định hướng phụ thuộc hoàn toàn vào tư thế hiện tại của khối. Bước đi bị coi là bất hợp lệ nếu bất kỳ phần nào của khối gạch rơi vào ô trống (void) hoặc ra ngoài mép bảng.
- **Điều kiện chiến thắng:** Trò chơi chỉ hoàn thành khi khối gạch rơi dựng đứng (standing upright) chính xác vào ô mục tiêu (Goal hole).
- **Các cơ chế & Ô đặc biệt (Advanced Tiles):**
  - **Ô dễ vỡ (Fragile tiles):** Chỉ chịu được trọng lực khi khối gạch nằm đè lên; nếu khối gạch đứng trên ô này, ô sẽ vỡ và khối gạch bị rơi.
  - **Cầu & Công tắc (Bridges & Switches):** Bao gồm công tắc nhẹ (Soft switch – kích hoạt khi nằm hoặc đứng), công tắc nặng (Heavy switch – chỉ kích hoạt khi đứng), và công tắc phân tách (Split switch – chia khối gạch thành hai khối $1 \times 1 \times 1$ hoạt động độc lập và hợp nhất lại khi kề nhau). Các công tắc này dùng để đóng/mở hệ thống cầu trên bản đồ.

1.2 Mục tiêu đồ án

Đồ án tập trung hoàn thành các mục tiêu cụ thể sau:

- **Mô hình hóa bài toán AI:** Định nghĩa không gian trạng thái S (bao gồm vị trí khối, định hướng và trạng thái đóng/mở của các cầu), tập hành động A, mô hình chuyển trạng thái $T(s, a)$, kiểm tra mục tiêu $G(s)$, và hàm chi phí $C(s, a, s')$.
- **Cài đặt các giải thuật tìm kiếm (Solvers):** Tự lập trình (không dùng thư viện thuật toán có sẵn) 4 giải thuật tìm kiếm bằng Python:
  - **BFS (Breadth-First Search):** Tìm kiếm theo chiều rộng.
  - **DFS / IDS (Depth-First Search / Iterative Deepening Search):** Tìm kiếm theo chiều sâu.
  - **UCS (Uniform Cost Search):** Tìm kiếm chi phí đồng nhất với hàm chi phí không đồng nhất tùy chỉnh.
  - **A* Search:** Tìm kiếm heuristic kết hợp hàm lượng giá đảm bảo tính tối ưu (Admissible) và nhất quán (Consistent).
- **Thực nghiệm & Đánh giá So sánh:** Kiểm thử các thuật toán trên bộ testcase chuẩn hóa phân theo độ khó tăng dần (màn cơ bản, màn có ô vỡ, màn có công tắc/cầu và màn kết hợp). Đo lường và so sánh dựa trên 4 chỉ số:

  - **Search Time:** Thời gian thực thi tìm kiếm.
  - **Memory Usage:** Dung lượng bộ nhớ đỉnh (Peak memory).
  - **Expanded Nodes:** Số lượng nút không gian trạng thái đã mở.
  - **Solution Length:** Độ dài lời giải (số bước di chuyển).

- **Phát triển Giao diện Đồ họa (GUI):** Xây dựng giao diện hiển thị cấu trúc màn chơi, trạng thái khối gạch, nút điều khiển và mô phỏng từng bước đi của lời giải.

1.3 Kế hoạch triển khai dự án

Đồ án được lên kế hoạch và thực hiện trong vòng 3 tuần với các mốc tiến độ cụ thể như sau:

- **Giai đoạn 0 (19/10 – 20/10) – Ổn định nền:**
  - *Mục tiêu:* Core import được và có test; GUI và BFS chạy được trên stage 1; mọi quyết định thiết kế được ghi lại.
- **Giai đoạn 1 (21/10 – 22/10) – Core đầy đủ và solver cơ bản:**
  - *Mục tiêu:* Core có đủ cơ chế nâng cao (fragile, cầu, 3 loại công tắc, tách/gộp khối); BFS, DFS chạy được; GUI vẽ đủ loại ô.
- **Giai đoạn 2 (23/10 – 25/10) – Đủ tính năng và chạy từ đầu đến cuối:**
  - *Mục tiêu:* 4 solver (BFS, DFS, UCS, A*) chạy được trên bộ map; GUI chọn màn, Solve, xem phát lại; script thực nghiệm chạy 1 lệnh.
- **Giai đoạn 3 (26/10 – 29/10) – Tích hợp, thực nghiệm chính thức**
  - *Mục tiêu:* Gộp mọi nhánh, chạy thực nghiệm lấy số liệu chính thức, đóng băng tính năng (tag `v1.0-rc`), hoàn thiện nháp báo cáo.
- **Giai đoạn 4 (28/10 – 29/10) – Báo cáo, video, đóng gói:**
  - *Mục tiêu:* Hoàn thiện `Report.pdf` (có link video); kiểm tra file zip nộp bài trên máy sạch; tập phỏng vấn.
- **Giai đoạn 5 (30/10 – 31/10) – Nộp bài:**
  - *Mục tiêu:* Nộp bài chính thức trước 18:00 ngày 30/10 (ngày 31/10 dự phòng).

1.4 Phân công nhiệm vụ & Kế hoạch chi tiết theo giai đoạn

1. Hoàng Trung Phong (Phụ trách Core Game Logic, State Transition, Replayer & Đóng gói)

- *Nhiệm vụ tổng quát:* Chịu trách nhiệm phát triển logic cốt lõi của game, quy tắc chuyển trạng thái (transition), cơ chế replay và đóng gói file nộp bài.
- *Triển khai theo giai đoạn:*
  - **Giai đoạn 0:** Sửa các lỗi core không import được, tạo `core/game.py`, dọn dẹp `board.py` và viết test kiểm tra các phép di chuyển/trạng thái.
  - **Giai đoạn 1:** Lập trình các cơ chế nâng cao: cầu toggle/permanent, công tắc soft/heavy, ô tách/gộp khối, đổi khối và cập nhật `map_loader`.

- **Giai đoạn 2:** Lưu dữ liệu người chơi, viết Replayer, xử lý chạy solver nền (thread), kiểm tra khóa trạng thái và viết phần mô hình hóa bài toán cho báo cáo (`docs/report_state_mod`
- **Giai đoạn 3:** Rà soát nhánh đã gộp, chốt `README.md` hướng dẫn chạy chi tiết, chuẩn hóa type hint và tên gọi trong core.
- **Giai đoạn 4:** Tạo cấu trúc thư mục nộp bài, đóng gói file zip (Source, `Report.pdf`, loại bỏ venv/__pycache__).

2. Hà Chí Tâm (Phụ trách Thuật toán Tìm kiếm BFS, DFS, UCS & Phân tích Đánh giá)

- *Nhiệm vụ tổng quát:* Xây dựng khung solver chung, cài đặt các thuật toán tìm kiếm mù (Uninformed Search), thuật toán UCS và phân tích thảo luận báo cáo.
- *Triển khai theo giai đoạn:*
  - **Giai đoạn 0:** Xây dựng `base_solver` (`SolveResult`, `BaseSolver` đo time, memory, nodes mở rộng), cài đặt BFS và DFS ban đầu.
  - **Giai đoạn 1:** Hoàn thiện BFS và DFS (thêm cơ chế visited, phát lại đường đi), thử nghiệm giải từ stage 0 đến stage 3.
  - **Giai đoạn 2:** Cài đặt UCS theo cost function, tối ưu băm trạng thái, chạy BFS/DFS/UCS trên toàn bộ map và viết mô tả thuật toán cho báo cáo (`docs/report_bfs_dfs_ucs.md`).
  - **Giai đoạn 3:** Phân tích kết quả thực nghiệm chính thức, viết nhận xét ưu/nhược điểm kèm số liệu (`docs/report_discussion.md`).
  - **Giai đoạn 4:** Đối chiếu toàn bộ số liệu báo cáo (đảm bảo khớp giữa báo cáo, bảng biểu và code), tham gia tập phỏng vấn.

3. Huỳnh Văn Phú (Phụ trách Heuristic, Thuật toán A* & Kiểm thử Chấp nhận/Máy sạch)

- *Nhiệm vụ tổng quát:* Xây dựng các hàm Heuristic, thuật toán A*, chứng minh tính chấp nhận được/nhất quán và kiểm thử độc lập file nộp.
- *Triển khai theo giai đoạn:*
  - **Giai đoạn 0:** Viết test case kiểm tra game logic ban đầu.
  - **Giai đoạn 1:** Chốt cost function, xây dựng các hàm Heuristic (Manhattan chia 2, phạt orientation, khoảng cách điểm, tách khối), kiểm tra tính chấp nhận được và bắt đầu viết A*.
  - **Giai đoạn 2:** Hoàn thiện A*, kiểm tra tính nhất quán, chạy trên bộ map và viết báo cáo thuật toán A* (`docs/report_astar.md`).
  - **Giai đoạn 3:** So sánh A* với UCS dựa trên số nút mở rộng, đánh giá chi tiết từng heuristic.
  - **Giai đoạn 4:** Giải nén file zip nộp bài trên máy sạch khác, tạo venv, cài `requirements.txt`, chạy `main.py` và `pytest` để ghi nhận lỗi.

4. Nguyễn Như Thuật (Phụ trách Giao diện GUI, Renderer, Control Panel & Kịch bản Video)

- *Nhiệm vụ tổng quát:* Thiết kế toàn bộ giao diện đồ họa (GUI), đồ họa các loại ô, hoạt cảnh và chuẩn bị danh sách cảnh quay video demo.

- *Triển khai theo giai đoạn:*
  - **Giai đoạn 0:** Sửa cửa sổ Window nhận Game, chạy stage 0, thêm nút New Game/Restart và điều khiển qua phím.
  - **Giai đoạn 1:** Đồ họa chi tiết toàn bộ các loại ô (sàn, đích, công tắc, cầu, ô tách khối), thêm hoạt cảnh lăn/rơi, HUD, thông báo thắng/thua.
  - **Giai đoạn 2:** Hoàn thiện Control Panel (nút Solve cho 4 thuật toán, chọn màn, bảng kết quả, solver chạy nền không treo GUI, vẽ hai khối đơn và đổi khối bằng phím SPACE).
  - **Giai đoạn 3:** Tinh chỉnh màu sắc/góc nhìn/animation, viết danh sách cảnh quay chi tiết cho video demo (`docs/video_script.md`).
  - **Giai đoạn 4:** Hỗ trợ tinh chỉnh giao diện cho video quay màn hình, tham gia tập phỏng vấn.

5. Nguyễn Hoàng Nam (Phụ trách Xây dựng Map, Script Thực nghiệm, Tổng hợp Báo cáo & Video YouTube)

- *Nhiệm vụ tổng quát:* Thiết kế bộ map thử nghiệm, viết pipeline đo đạc/vẽ biểu đồ, tổng hợp file `Report.pdf` và quay/dựng video công khai YouTube.
- *Triển khai theo giai đoạn:*
  - **Giai đoạn 0:** Tạo các map nhóm E, B, D, permanent bridge, lập `data/maps/index.md`, viết `test_map_loader` và setup khung repository/requirements.
  - **Giai đoạn 1:** Thiết kế thêm map mới, tạo khung `run_experiments.py` & CSV schema, lập dàn ý báo cáo và bảng % đóng góp nháp.
  - **Giai đoạn 2:** Chạy thử pipeline thực nghiệm, viết `plot_results.py` (vẽ 4 biểu đồ), tổng hợp prompt AI của các thành viên.
  - **Giai đoạn 3:** Chạy thực nghiệm chính thức trên máy cố định (3 lần/cặp map-solver lấy trung vị), xuất bảng/biểu đồ, viết phần thực nghiệm (`docs/report_experiments.md`).
  - **Giai đoạn 4:** Ghép các phần thành `Report.pdf` hoàn chỉnh (kế hoạch, % đóng góp, tài liệu tham khảo, phụ lục GenAI), quay màn hình/lồng tiếng và đăng video lên YouTube.

6. Cả nhóm

- **Giai đoạn 0:** Họp chia vai trò, thống nhất các quy ước làm việc.
- **Giai đoạn 3:** Sáng 26/10 gộp tất cả các nhánh, chạy toàn bộ test, sửa lỗi; tối 27/10 đóng băng tính năng và gắn tag `v1.0-rc`.
- **Giai đoạn 4:** Họp 60 phút tập phỏng vấn (mỗi thành viên giải thích module của người khác và trả lời bộ câu hỏi mẫu).
- **Giai đoạn 5:** Nộp bài chính thức, tải lại file nộp để kiểm tra.

1.5 Bảng tỷ lệ đóng góp công việc

STT	Thành viên	Phân tích & thiết kế	Lập trình	Thực nghiệm	Làm báo cáo, video	Tỉ lệ chung
1	Hoàng Trung Phong	30%	40%	15%	15%	**20%**
2	Huỳnh Văn Phú	15%	55%	15%	15%	**20%**
3	Hà Chí Tâm	20%	50%	15%	15%	**20%**
4	Nguyễn Như Thuật	20%	50%	15%	15%	**20%**
5	Nguyễn Hoàng Nam	10%	15%	35%	40%	**20%**

Bảng 1: Bảng chi tiết ma trận phân công và tỉ lệ đóng góp công việc của các thành viên.

2 Kiến trúc luồng trò chơi, chức năng các thành phần hệ thống và hướng dẫn sử dụng

2.1 Quy trình vận hành trò chơi

2.1.1 Luồng tương tác và chuyển trạng thái hệ thống

Dựa trên sơ đồ thiết kế hệ thống, luồng hoạt động của trò chơi được chia thành các giai đoạn chính từ khi khởi tạo, tương tác của người chơi cho đến khi xử lý chuyển màn.

2.1.2 Khởi tạo hệ thống

Quá trình vận hành bắt đầu tại tệp `main.py`.

- Hệ thống lần lượt khởi tạo danh mục màn chơi bằng `LevelCatalog.from_json(coolmath.json)`, trình quản lý dữ liệu người chơi `PlayerDataManager`, thực thể trò chơi `Game(catalog.board(0))` và giao diện cửa sổ `Window`.

- Tiếp theo, phiên chơi `GameSession` được thiết lập. Ở trạng thái ban đầu, hệ thống gọi `catalog.unlock` để mở khóa màn đang chơi (mặc định chỉ có Stage 01 được mở).

2.1.3 Luồng tương tác của người chơi

Trong quá trình chơi, người chơi có thể tạo ra các luồng rẽ nhánh trạng thái khác nhau:

a. Xử lý chiến thắng (Hoàn thành màn chơi):

- *Tự qua màn (Thủ công):* Nếu người chơi tự giải quyết màn chơi, hệ thống gọi `GameSession._on_win` để ghi nhận kỷ lục ở chế độ `MANUAL`. Màn chơi kế tiếp tự động được mở khóa qua lệnh `catalog.unlock(màn kế tiếp)`. Một băng rôn chiến thắng sẽ hiện lên thông báo tên người chơi và mật khẩu của màn kế. Khi người chơi nhấn `Enter`, hệ thống chuyển tới màn tiếp theo thông qua `Window._advance_to_next`.

- *Nhờ AI hỗ trợ giải (Assisted):* Nếu hoàn thành màn chơi nhờ sự trợ giúp của thuật toán giải, hệ thống sẽ không ghi nhận kỷ lục và không mở khóa màn kế tiếp. Băng rôn sẽ hiển thị thông báo màn kế chưa được mở và yêu cầu người chơi nhấn phím `R` để chơi lại màn hiện tại một cách hợp lệ.

b. Chọn màn chơi (Phím N):

- Khi nhấn phím `N` hoặc nút "Chọn màn", giao diện `LevelPickerOverlay` xuất hiện hiển thị danh sách 33 màn chơi cùng với các đánh dấu (đang chơi) hoặc (khóa).
- Hệ thống gọi `GameSession.select_stage(index)` và trả về một trạng thái `StageSelection`:
  - Nếu là `CURRENT` hoặc `INVALID`: Hệ thống đóng hộp thoại, giữ nguyên ván cờ hiện tại.
  - Nếu là `LOADED` (màn đã mở): Lập tức gọi lệnh `GameSession.load_stage(index)` để nạp màn.
  - Nếu là `NEEDS_PASSCODE` (màn bị khóa): Kích hoạt giao diện `PasscodeOverlay` yêu cầu người dùng nhập 6 chữ số.

c. Xử lý nhập mật khẩu (PasscodeOverlay):

- Nếu nhấn phím `Esc`, hệ thống quay lại giao diện chọn màn (`LevelPickerOverlay`).
- Nếu nhấn `Enter` nhưng nhập thiếu chữ số, hệ thống báo lỗi "mật khẩu gồm 6 chữ số" và yêu cầu nhập lại.
- Nếu nhập đủ số và nhấn `Enter`, hệ thống chạy `GameSession.submit_passcode(index, text)` và kiểm tra bằng `catalog.unlock_with_passcode` để so khớp dữ liệu chính xác (không dựa vào số thứ tự màn).
- Nếu mã đúng, màn chơi được mở khóa (`catalog.unlock(index)`) và tiến hành nạp màn. Nếu sai, hệ thống báo lỗi sai mật khẩu, xóa ô nhập và yêu cầu nhập lại.

d. Khởi động lại màn chơi (Phím R):

- Khi nhấn `R`, hàm `GameSession.restart()` được gọi. Trạng thái ván cờ được làm mới ở cùng màn chơi hiện tại và cờ `assisted` bị ép đặt về `False`.

2.1.4 Tiến trình nạp màn (Load Stage)

Khi hệ thống xác nhận nạp màn mới qua lệnh `Game(catalog.board(index))`, quy trình sau diễn ra:

- **Nạp thất bại:** Hệ thống báo lỗi nạp màn và giữ nguyên ván cờ hiện tại.
- **Nạp thành công:** Biến đếm `session.revision` được cộng thêm 1. Hàm `Window._sync_stage` được gọi để dọn dẹp hệ thống: hủy tiến trình giải AI, bỏ trạng thái phát lại, hủy bộ vẽ cũ (`Renderer.dispose()`) để tạo bộ vẽ mới, và gọi `ScenePresenter.reset`.
- Cuối cùng, hệ thống đặt lại đồng hồ đếm thời gian, xóa các thông báo/băng rôn cũ, khôi phục cờ `assisted` và làm mới trạng thái các ô gạch vỡ trước khi trao lại quyền điều khiển cho người chơi.

Quá trình vận hành trò chơi bắt đầu từ tệp `main.py`, tại đây hệ thống khởi tạo các thành phần cốt lõi bao gồm `LevelCatalog.from_json(coolmath.json)`, `PlayerDataManager`, `Game` và lớp `Window`. Từ các đối tượng này, hệ thống tiếp tục khởi tạo `GameSession`, trong đó mặc định ban đầu chỉ có màn đầu tiên (Stage 01) được mở khóa.

Dựa vào sơ đồ trên, quy trình xử lý và chuyển trạng thái của hệ thống được quản lý tuần tự qua các thành phần:

- **LevelPickerOverlay (Giao diện chọn màn)**:
  - Giao diện này được mở khi người chơi nhấn phím `N` hoặc bấm "nút Chọn màn".

  - Hệ thống hiển thị danh sách 33 màn chơi, đồng thời đánh dấu trạng thái màn đang chơi và các màn bị khóa.
  - Khi người chơi chọn một màn, hệ thống gọi `GameSession.select_stage(index)` và trả về một trong các trạng thái `StageSelection`.
  - Nếu trạng thái trả về là `CURRENT` hoặc `INVALID`, hệ thống sẽ đóng hộp thoại và giữ nguyên ván cờ hiện tại.
  - Nếu trạng thái là `LOADED` (màn đã được mở khóa), hệ thống tiến hành nạp màn bằng `GameSession.load_stage(index)`.
  - Nếu trạng thái là `NEEDS_PASSCODE` (màn đang khóa), hệ thống kích hoạt giao diện `PasscodeOverlay`.

- **PasscodeOverlay (Giao diện nhập mật khẩu)**:
  - Yêu cầu người dùng nhập mật khẩu gồm 6 chữ số và chỉ ghi nhận ký tự chữ số.
  - Nếu người dùng nhấn `Enter` nhưng nhập thiếu chữ số, hệ thống sẽ báo lỗi mật khẩu phải gồm đủ 6 chữ số và chờ nhập lại.
  - Khi nhập đủ và nhấn `Enter`, hệ thống gọi `GameSession.submit_passcode(index, text)` để so khớp chính xác qua hàm `catalog.unlock_with_passcode`.
  - Nếu đúng mật khẩu, hệ thống gọi `catalog.unlock(index)` và chuyển sang tiến trình nạp màn. Nếu sai, hiển thị thông báo lỗi, xóa ô nhập và chờ người dùng thao tác lại.

- **Tiến trình nạp màn (Load Stage)**:
  - Dữ liệu màn chơi được nạp thông qua `Game(catalog.board(index))`.
  - Nếu nạp thành công, biến đếm `session.revision` được cộng thêm 1. Hệ thống gọi `Window._sync_stage` để dọn dẹp: hủy thuật toán giải AI, bỏ chế độ phát lại, tạo `Renderer` mới, đặt lại đồng hồ và các thông báo.

2.1.5 Cơ chế thực thi chế độ chơi & Xem phát lại lời giải

Hệ thống điều hướng luồng thực thi và lưu trữ kết quả khác nhau dựa trên phương thức hoàn thành màn chơi của người dùng:

- **Chế độ chơi thủ công (Người chơi tự thắng)**:
  - Kết quả được ghi nhận vào kỷ lục lưu trữ với trạng thái `MANUAL` thông qua hàm `GameSession._on_win`.
  - Màn chơi kế tiếp sẽ tự động được mở khóa (`catalog.unlock(màn kế tiếp)`).
  - Giao diện hiển thị băng chúc mừng chiến thắng, nêu tên người chơi và mật khẩu của màn kế tiếp. Sau đó, người chơi nhấn `Enter` để kích hoạt `Window._advance_to_next` và tiến sang màn chơi tiếp theo.

- **Chế độ nhờ AI giải (Assisted)**:
  - Nếu màn chơi được vượt qua nhờ vào thuật toán giải tự động (`assisted`), hệ thống sẽ không ghi lại kỷ lục và không mở khóa màn kế tiếp.
  - Băng thông báo sẽ xuất hiện với nội dung nêu rõ màn kế tiếp chưa được mở, yêu cầu người chơi nhấn phím `R` để khởi động và chơi lại màn đó.

- **Phát lại (Replay)**:

- Khi người chơi nhấn phím `R`, hệ thống lập tức gọi `GameSession.restart()`.
- Hành động này nạp lại cùng một màn chơi hiện tại, buộc đặt lại cờ `assisted = False`, và khôi phục toàn bộ giao diện về trạng thái khởi điểm.

2.2 Kiến trúc và Chức năng các Thành phần Hệ thống

Hệ thống được thiết kế theo kiến trúc phân tầng rõ ràng, đảm bảo tính đóng gói, dễ bảo trì và mở rộng. Chức năng cụ thể của các thành phần chính bao gồm:

1. **Thành phần Cốt lõi (Core Engine - `core/`):** Chịu trách nhiệm quản lý toàn bộ logic và quy tắc vận hành của trò chơi:
   - **Mô hình hóa trạng thái (`game.py`):** Biểu diễn trạng thái khối gạch (vị trí, định hướng, trạng thái phân tách) và trạng thái đóng/mở của hệ thống cầu.
   - **Cơ chế chuyển trạng thái (`board.py`):** Xử lý các phép lăn, tính toán tọa độ mới, kiểm tra tính hợp lệ của nước đi (rơi vực, ô vỡ) và kích hoạt các loại công tắc (Soft, Heavy, Split).
   - **Đọc dữ liệu màn chơi (`map_loader.py`):** Nạp và giải mã cấu trúc bản đồ từ file cấu hình JSON/text.
2. **Giao diện Người dùng (GUI - `gui/`):** Đảm nhận việc tương tác trực quan với người chơi thông qua thư viện Pygame:
   - **Bộ dựng hình (`renderer.py`):** Vẽ lưới bản đồ, khối gạch 3D/2D, hiệu ứng lăn/rơi và các loại ô đặc biệt.
   - **Bảng điều khiển (`control_panel.py`):** Hỗ trợ người dùng chọn màn chơi, kích hoạt các thuật toán AI giải tự động, xem bảng thống kê kết quả và điều khiển phát lại (replay).
   - **Lớp phủ giao diện (`overlays.py`):** Hiển thị hộp thoại chọn màn (`LevelPicker`), nhập mật khẩu (`PasscodeOverlay`) và thông báo chiến thắng.
3. **Bộ giải thuật Tìm kiếm (Solvers - `algorithms/`):** Chứa các thuật toán Trí tuệ Nhân tạo để tìm lời giải tối ưu:
   - `base_solver.py`**:** Định nghĩa khung làm việc chung, đo đạc thời gian thực thi, dung lượng bộ nhớ đỉnh (`tracemalloc`) và số nút đã mở rộng.
   - `bfs.py`**,** `dfs.py`**,** `ucs.py`**:** Cài đặt các giải thuật tìm kiếm mù (Breadth-First, Depth-First/IDS, Uniform Cost Search).
   - `astar.py` **&** `heuristics.py`**:** Cài đặt thuật toán A^* kết hợp các hàm lượng giá Heuristic (H_1, H_2) giúp tối ưu tốc độ tìm kiếm.
4. **Trình phát lại & Quản lý (Replayer & Data Manager):**
   - `replayer.py`**:** Lưu trữ chuỗi hành động và tự động mô phỏng lại từng bước di chuyển của lời giải trên giao diện.
   - `player_data.py`**:** Quản lý tiến trình chơi, lưu trữ kỷ lục thời gian/số bước và trạng thái mở khóa các màn chơi.

2.3 Hướng dẫn cài đặt & chạy chương trình

2.3.1 Yêu cầu môi trường & Tải mã nguồn

Dự án được phát triển và kiểm thử trên môi trường Python (3.9+), hỗ trợ đa nền tảng (Windows, Linux, macOS).

1. **Cài đặt Python (3.9+):**
   - *Trên Windows:* Tải bộ cài chính thức tại `python.org`. **Lưu ý:** Bắt buộc tích chọn vào ô `"Add Python to PATH"` trong quá trình cài đặt.
   - *Trên Linux (Ubuntu/Debian):* Mở Terminal và chạy lệnh sau để cài đặt Python cùng công cụ tạo môi trường ảo:

```
sudo apt update
sudo apt install python3 python3-pip python3-venv -y
```

2. **Tải mã nguồn từ GitHub:** Sử dụng Git để clone mã nguồn dự án về máy và di chuyển vào thư mục gốc của đồ án:

```
git clone https://github.com/ng-h-nam/bloxorz-solver.git
cd bloxorz-solver
```

2.3.2 Thiết lập môi trường ảo (Virtual Environment)

Khuyến khích sử dụng môi trường ảo để tránh xung đột các thư viện với hệ thống:

- **Trên Windows (CMD / PowerShell):**

```
# Tao moi truong ao ten la .venv
python -m venv .venv

# Kich hoat moi truong ao
.venv\Scripts\activate
```

- **Trên Linux / macOS:**

```
# Tao moi truong ao ten la .venv
python3 -m venv .venv

# Kich hoat moi truong ao (dau dong lenh se xuat hien (.venv))
source .venv/bin/activate
```

Mẹo: Khi muốn thoát khỏi môi trường ảo trên cả hai hệ điều hành, bạn chỉ cần gõ lệnh `deactivate`.

2.3.3 Cài đặt thư viện phụ thuộc & Khởi chạy chương trình

1. **Cài đặt các gói thư viện:** Cập nhật trình quản lý gói `pip` lên phiên bản mới nhất và cài đặt các thư viện yêu cầu từ file cấu hình:

```
pip install --upgrade pip
pip install -r requirements.txt
```

2. **Khởi chạy ứng dụng:**

   - *Trên Windows:*

```
python main.py
```

   - *Trên Linux / macOS:*

```
# Khi da kich hoat moi truong ao (.venv), co the dung python
   hoac python3
python3 main.py
```

3 Biểu diễn không gian trạng thái & cơ chế trò chơi

3.1 Biểu diễn trạng thái

Trò chơi Bloxorz được mô hình hóa thành một bài toán tìm kiếm, trong đó không gian trạng thái được biểu diễn thông qua một bộ 6 thành phần bất biến $\langle r, c, \theta, \mathcal{B}, p_{\text{split}}, a_{\text{cube}} \rangle$. Các thành phần này bao gồm:

- $(r, c) \in \mathbb{Z}^2$: Tọa độ điểm neo (hàng r, cột c) của khối, hoặc tọa độ của khối đơn thứ nhất khi bị phân tách.
- $\theta \in \{\text{STANDING}, \text{HORIZONTAL}, \text{VERTICAL}, \text{SPLIT}\}$: Tư thế hiện tại của khối.
- $\mathcal{B} \subseteq \mathbb{Z}^2$: Tập hợp bất biến (`frozenset`) lưu tọa độ các ô cầu đang ở trạng thái mở.
- $p_{\text{split}} \in \mathbb{Z}^2 \cup \{\text{None}\}$: Tọa độ (r_2, c_2) của khối đơn thứ hai khi $\theta = \text{SPLIT}$.
- $a_{\text{cube}} \in \{0, 1\}$: Chỉ số khối đơn đang được chọn điều khiển tuần tự khi bị tách khối.

Hệ thống áp dụng tiên đề điểm neo trên cùng - bên trái (Top-Left Anchor Invariant) để đảm bảo (r, c) luôn là ô có chỉ số hàng và cột nhỏ nhất, giúp triệt tiêu hoàn toàn sai số tích lũy tọa độ khi khối lăn. Để các thuật toán tìm kiếm có thể so sánh trạng thái hiệu quả với độ phức tạp $\mathcal{O}(1)$, khóa băm trạng thái (`state_key`) được định nghĩa như sau:

$$\text{state_key}(s) = \Big(r,\ c,\ \theta.\text{value},\ \tilde{p}_{\text{split}},\ a_{\text{cube}},\ \text{tuple}\big(\text{sorted}(\mathcal{B})\big) \Big)$$

3.2 Không gian Hành động & bảng chuyển trạng thái

Mỗi hành động lăn $d \in \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$ áp dụng lên khối (khi chưa bị tách) được định nghĩa qua bảng chuyển trạng thái hình học `TRANSITION_TABLE`. Mỗi phép lăn thực hiện ánh xạ đơn định sang $(\Delta r, \Delta c, \theta')$ trong thời gian $\mathcal{O}(1)$.

Tư thế hiện tại	Hướng lăn	Độ lệch	Tư thế mới	Các ô chiếm giữ mới
STANDING	UP	$(-2, 0)$	VERTICAL	$\{(r-2, c), (r-1, c)\}$
STANDING	DOWN	$(+1, 0)$	VERTICAL	$\{(r+1, c), (r+2, c)\}$
STANDING	LEFT	$(0, -2)$	HORIZONTAL	$\{(r, c-2), (r, c-1)\}$
STANDING	RIGHT	$(0, +1)$	HORIZONTAL	$\{(r, c+1), (r, c+2)\}$
HORIZONTAL	UP	$(-1, 0)$	HORIZONTAL	$\{(r-1, c), (r-1, c+1)\}$
HORIZONTAL	DOWN	$(+1, 0)$	HORIZONTAL	$\{(r+1, c), (r+1, c+1)\}$
HORIZONTAL	LEFT	$(0, -1)$	STANDING	$\{(r, c-1)\}$
HORIZONTAL	RIGHT	$(0, +2)$	STANDING	$\{(r, c+2)\}$
VERTICAL	UP	$(-1, 0)$	STANDING	$\{(r-1, c)\}$
VERTICAL	DOWN	$(+2, 0)$	STANDING	$\{(r+2, c)\}$
VERTICAL	LEFT	$(0, -1)$	VERTICAL	$\{(r, c-1), (r+1, c-1)\}$
VERTICAL	RIGHT	$(0, +1)$	VERTICAL	$\{(r, c+1), (r+1, c+1)\}$

Bảng 2: Bảng 12 phép lăn cơ bản và độ lệch tọa độ.

Phép biến đổi này thỏa mãn tính chất bất biến đối xứng (Round-trip Invariant), nghĩa là lăn đi và lăn ngược lại hướng ban đầu luôn trở về đúng trạng thái hình học cũ.

3.3 Kiểm tra nước đi Hợp lệ & điều kiện Thắng

Việc kiểm tra tính hợp lệ của một trạng thái ứng viên trải qua 3 bước phân rã nghiêm ngặt:

1. **Vượt biên:** Kiểm tra mọi ô đang chiếm giữ có nằm trong giới hạn $R \times C$ của lưới không gian hay không.
2. **Kiểm tra vực (Void):** Đảm bảo không có bất kỳ phần nào của khối đè lên ô trống (‘TileType.VOID‘).
3. **Kiểm tra loại gạch hợp lệ:** Đối chiếu với các ràng buộc của cầu đang đóng/mở và tính chất chịu lực của gạch mỏng.

Trạng thái đích (Thắng) được định nghĩa là khi khối đứng thẳng đứng (`STANDING`) tại ô mục tiêu $T(r_{\text{end}}, c_{\text{end}})$.

3.4 Logic tương tác các loại ô đặc biệt

3.4.1 Ô giòn (Fragile Tiles)

Ô gạch mỏng màu cam (`FRAGILE`) chỉ chịu được lực phân tán khi khối nằm ngang, nằm dọc hoặc đã được tách thành các khối đơn. Nếu khối gạch dựng đứng (`STANDING`) trên ô này, gạch sẽ vỡ ngay lập tức và nước đi bị từ chối do rơi khối.

3.4.2 Công tắc (Switches: Soft Switch, Heavy Switch, Split Switch)

- **Soft Switch:** Kích hoạt ngay khi bất kỳ phần nào của khối gạch đè lên, bất kể tư thế hay khối đã bị tách.
- **Heavy Switch:** Chỉ kích hoạt khi khối gạch nguyên vẹn đứng thẳng (`STANDING`) trên đó.
- **Split Switch:** Khi khối đứng thẳng trên ô này, nó lập tức dịch chuyển tới 2 vị trí định sẵn và chuyển sang tư thế `SPLIT`.

3.4.3 Cầu nối (Bridges)

Tập hợp cầu đang mở $\mathcal{B}$ sẽ được cập nhật thông qua 3 chế độ hoạt động:

- **TOGGLE:** Hoán đổi trạng thái cầu giữa mở và đóng.
- **OPEN:** Mở vĩnh viễn cây cầu liên kết.
- **CLOSE:** Đóng vĩnh viễn cây cầu liên kết.

Sau khi công tắc cập nhật trạng thái cầu, hệ thống sẽ gọi hậu kiểm tra để chặn trường hợp khối đang đứng bị rơi do kích hoạt công tắc làm sập chính cây cầu bên dưới.

4 Các thuật toán sử dụng

4.1 Thuật toán Breadth-First Search (BFS)

Thuật toán duyệt không gian trạng thái theo từng lớp độ sâu (level-by-level), trong đó nút sinh trước sẽ được mở rộng trước. Cấu trúc dữ liệu hàng đợi (Frontier) được cài đặt bằng `collections.deque` giúp các thao tác thêm và lấy phần tử đạt độ phức tạp $O(1)$.

Hệ thống áp dụng kỹ thuật **Early Goal Test**, kiểm tra đích ngay khi sinh trạng thái con trước khi đưa vào hàng đợi. Cơ chế này giúp giảm từ 20% đến 45% số lượng nút cần sinh và duyệt, tiết kiệm đáng kể thời gian và bộ nhớ. Về mặt lý thuyết, BFS đảm bảo tính đầy đủ và tính tối ưu về số bước đi khi chi phí là hằng số, với độ phức tạp không gian là $O(b^d)$.

4.2 Thuật toán Depth-First Search, Iterative Deepening Search

4.2.1 Nguyên lý hoạt động & Giới hạn độ sâu

DFS duyệt sâu nhất có thể dọc theo mỗi nhánh trước khi quay lui, sử dụng cấu trúc LIFO thông qua `list.append` và `list.pop`. Ưu điểm lớn nhất là bộ nhớ tiêu thụ rất nhỏ, chỉ $O(b \cdot m)$. Tuy nhiên, DFS không đảm bảo tính tối ưu của đường đi. Để khắc phục việc DFS đi vào nhánh vô hạn, một giới hạn độ sâu `max_depth` được thiết lập để cắt tỉa (pruning) nhánh tìm kiếm.

Iterative Deepening Search (IDS) kết hợp ưu điểm bằng cách lặp lại việc gọi DFS với giới hạn độ sâu tăng dần. Nhờ đó, IDS đạt được tính đầy đủ và tính tối ưu như BFS nhưng vẫn duy trì độ phức tạp không gian tuyến tính $O(b \cdot d)$ giống DFS.

4.2.2 Cơ chế lưu tập trạng thái đã duyệt (Visited Set / Hashing state key)

Để tránh vòng lặp kín (Graph Search), mô hình trạng thái được chuẩn hóa bằng hàm `to_key()` đảm bảo các trạng thái cầu được sắp xếp để tạo ra khóa băm (`hash`) duy nhất. Đối với IDS, thay vì chỉ lưu một tập hợp (`set`), thuật toán sử dụng một từ điển `visited_depth` để lưu độ sâu nhỏ nhất từng đạt tới trạng thái đó, từ đó quyết định có duyệt lại hay không nếu tìm thấy một đường đi ngắn hơn.

4.3 Thuật toán Uniform Cost Search

4.3.1 Cấu trúc Priority Queue

UCS mở rộng nút có chi phí đường đi tích lũy nhỏ nhất bằng cách sử dụng hàng đợi ưu tiên Min-Heap (`heapq`). Mỗi phần tử trong heap được lưu dưới dạng bộ ba `(priority_cost, tie_counter, node)`. Biến `tie_counter` dùng để tránh lỗi so sánh giữa các đối tượng và đảm

bảo tính ổn định FIFO khi chi phí bằng nhau. Thuật toán áp dụng **Pop-Time Goal Test** và kỹ thuật Lazy Deletion kết hợp từ điển `best_g` để bỏ qua các trạng thái có đường đi chi phí rẻ hơn đã được xử lý.

4.3.2 Đề xuất Thiết kế Non-uniform Cost Function

Ba hàm chi phí không đồng nhất được đề xuất để thay đổi hành vi tìm kiếm:

- **Phạt đổi trạng thái cầu:** Thêm chi phí khi giẫm lên công tắc nhằm khuyến khích tìm đường đi ổn định, giảm rủi ro sập cầu.
- **Phạt ô mỏng và mép vực:** Tăng chi phí đi qua gạch mỏng hoặc sát vực thẳm để khối di chuyển an toàn hơn ở trung tâm bản đồ.
- **Phạt động học quán tính:** Ưu tiên di chuyển theo đà (chi phí rẻ), phạt khi bẻ góc 90°, và đánh giá chi phí đổi khối (`SWITCH`) rất thấp.

4.4 Thuật toán A* Search

4.4.1 Nguyên lý thuật toán & Cài đặt (`algorithms/astar.py`)

Lớp `AStarSolver` trong thư mục `algorithms/astar.py` được cài đặt tối ưu với các cơ chế chính sau:

- **Cấu trúc dữ liệu & Hàng đợi ưu tiên (Priority Queue):** Sử dụng thư viện `heapq` với mỗi phần tử lưu trữ dưới dạng một tuple:

$$(f_{\text{score}}, h_{\text{cost}}, \text{tie_counter}, \text{node})$$

  Biến `tie_counter` là một bộ đếm tăng dần tự động, đóng vai trò tiêu chí giải quyết hòa (*tie-breaking*) khi hai nút có cùng giá trị f và h.

- **Cơ chế cắt tỉa và lưu chi phí tốt nhất (`best_g`):** Thuật toán sử dụng một từ điển `best_g[state_key]` để lưu lại chi phí g nhỏ nhất từng đạt được tới mỗi trạng thái. Trong quá trình duyệt, nếu phát hiện một trạng thái đã từng đến với $g_{\text{new}} \geq g_{\text{old}}$, thuật toán sẽ lập tức bỏ qua để cắt tỉa nhánh tìm kiếm.

- **Kiểm tra trạng thái đích tại thời điểm Pop (*Pop-time Goal Test*):** Thuật toán chỉ thực hiện kiểm tra điều kiện đích (`is_goal`) ngay khi nút được lấy ra khỏi Frontier (`heappop`) thay vì lúc vừa sinh nút con, giúp bảo đảm tuyệt đối tính tối ưu của đường đi.

- **Đo lường hiệu năng thực thi:** Sử dụng `time.perf_counter()` để đo chính xác thời gian tìm kiếm và mô-đun `tracemalloc` để đo lượng bộ nhớ RAM đỉnh (*Peak Memory*) tiêu thụ.

4.4.2 Đề xuất thiết kế các hàm Heuristic

- **Xác định hàm chi phí $g(n)$:** Tất cả các hành động di chuyển hợp lệ $a \in \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}$ đều có chi phí bằng 1:

$$c(s, a, s') = 1 \quad \forall s, a, s' \implies g(n) = \text{Tổng số bước di chuyển từ gốc}$$

- **Heuristic cơ sở h_1 (`manhattan_heuristic` - Baseline):** Sử dụng khoảng cách Manhattan tối thiểu từ các ô mà khối đang chiếm giữ đến ô đích $T(r_T, c_T)$:

$$d_{\min}(s, T) = \min_{(r,c)\in \text{cells}(s)} (|r - r_T| + |c - c_T|)$$

Công thức Heuristic cơ sở:

$$h_1(s) = \left\lceil \frac{d_{\min}(s, T)}{2} \right\rceil$$

- **Heuristic nâng cao h_2 (`bloxorz_advanced_heuristic`):** Bổ sung phạt tư thế và xử lý trạng thái tách khối:

1. *Khoảng cách điểm rút gọn:* Đặt mốc chi phí nền base_h = $\lceil \frac{d_{\min}}{2} \rceil$.
2. *Phạt tư thế (Orientation Penalty):* Do ô đích T bắt buộc khối phải ở tư thế đứng (`STANDING`):
   - **Khối nằm đè lên ô đích** ($d_{\min} = 0$ và $\theta \neq$ STANDING): Cần tối thiểu 2 bước lăn để dựng lại vào đích. Áp dụng phạt an toàn $+2 \implies h(s) = 2$.
   - **Khối đứng cạnh ô đích** ($d_{\min} = 1$ và $\theta =$ STANDING): Cần tối thiểu 2 bước nữa để quay lại đứng trên đích. Áp dụng phạt an toàn $+1 \implies h(s) = 2$.
3. *Trạng thái phân tách khối (`SPLIT`):* Khi khối bị tách thành 2 khối đơn $1 \times 1 \times 1$ tại $C_1(r_1, c_1)$ và $C_2(r_2, c_2)$, hàm tính toán chi phí gộp khối d_{merge} và khoảng cách từ khối kề nhất tới đích d_{goal}:

$$d_{\text{merge}} = \max(0, |r_1 - r_2| + |c_1 - c_2| - 1)$$

$$d_{\text{goal}} = \left\lceil \frac{\min(|C_1 - T|_1, |C_2 - T|_1)}{2} \right\rceil$$

$$h_{\text{split}}(s) = \max(d_{\text{merge}}, d_{\text{goal}})$$

4.4.3 Chứng minh tính Admissibility và Consistency

- **Tính chấp nhận được (Admissibility):** Heuristic $h(s)$ thỏa mãn $\forall s \in \mathcal{S}, 0 \leq h(s) \leq h^*(s)$. Vì vận tốc dịch chuyển tối đa của khối trên bàn cờ là 2 ô/bước, ước lượng $h_1(s) = \lceil d_{\min}/2 \rceil$ là cận dưới chắc chắn không vượt quá chi phí thực tế $h^*(s)$. Các mức phạt $+1$ và $+2$ dựa trên số bước tối thiểu bắt buộc phải thực hiện để chuyển đổi tư thế về `STANDING` tại đích nên vẫn đảm bảo tính Admissible.

- **Kiểm chứng thực nghiệm (`tests/test_heuristic.py`):** Chạy thuật toán duyệt toàn bộ không gian trạng thái trên các map nhỏ (Level 0, Level 1, Level 3) đã xác nhận kết quả **0 lỗi vi phạm admissibility** ($h(s) \leq h^*(s)$ tuyệt đối trên 100% trạng thái).

5 Thực nghiệm và đánh giá

5.1 Thiết kế bộ test cases

Bộ dữ liệu chuẩn (benchmark) để đánh giá các thuật toán được thiết kế cố định gồm 22 màn chơi. Thay vì sinh ngẫu nhiên, các màn chơi này được phân loại thành 4 nhóm với độ khó và cơ chế tăng dần:

- **Cơ bản (3 màn):** Lưới chỉ bao gồm sàn, ô trống và hố đích.

- **Ô giòn (5 màn):** Bổ sung từ 6 đến 13 ô giòn, đòi hỏi khối chỉ được nằm ngang khi di chuyển qua.
- **Công tắc (8 màn):** Giới thiệu cơ chế cầu và các loại công tắc (soft, heavy, split). Đặc biệt, cơ chế split yêu cầu điều khiển hai khối lập phương riêng biệt.
- **Kết hợp (6 màn):** Đòi hỏi xử lý đồng thời ô giòn, cầu và công tắc, với độ dài lời giải lên tới 39 bước.

Số bước đi tối ưu của toàn bộ 22 màn chơi đều được tính toán và kiểm chứng tính hợp lệ thông qua thuật toán BFS.

Màn	Nhóm	Cỡ	Fragile	Bridge	Switch	Tối ưu
Stage 01	Cơ bản	6x10	0	0	0	7
Stage 02	Cơ bản	6x15	0	4	2	17
Stage 03	Cơ bản	6x15	0	0	0	19
Stage 04	Cơ bản	9x14	31	0	0	28
Stage 05	Cơ bản	10x15	6	6	4	33
Stage 06	Cơ bản	10x15	0	0	0	35
Stage 07	Cơ bản	8x15	0	1	1	44
Stage 08	Cơ bản	9x15	0	0	0	11
Stage 09	Cơ bản	5x15	0	0	0	25
Stage 10	Cơ bản	10x14	0	6	2	58
Stage 11	Cơ bản	10x12	2	2	1	47
Stage 12	Cơ bản	10x13	0	2	2	65
Stage 13	Cơ bản	10x14	22	0	0	46
Stage 14	Cơ bản	10x14	0	4	2	67
Stage 15	Cơ bản	10x15	4	8	4	61
Stage 16	Cơ bản	8x13	0	4	2	33
Stage 17	Cơ bản	10x15	0	4	5	106
Stage 18	Cơ bản	9x15	0	7	6	85
Stage 19	Cơ bản	10x15	2	4	3	67
Stage 20	Cơ bản	10x15	2	6	5	58
Stage 21	Cơ bản	10x15	0	2	2	71
Stage 22	Cơ bản	10x14	1	3	4	65
Stage 23	Cơ bản	10x15	21	8	4	75
Stage 24	Cơ bản	8x14	0	6	4	57
Stage 25	Cơ bản	10x14	4	9	3	55
Stage 26	Cơ bản	9x14	2	4	2	106
Stage 27	Cơ bản	10x15	29	2	3	71
Stage 28	Cơ bản	10x15	11	4	1	101
Stage 29	Cơ bản	10x15	6	17	6	104
Stage 30	Cơ bản	10x15	42	5	3	114
Stage 31	Cơ bản	10x15	15	14	5	91
Stage 32	Cơ bản	9x14	2	8	3	129
Stage 33	Cơ bản	10x15	4	5	13	65

5.2 Phương pháp đo lường

Quá trình thực nghiệm được triển khai trên môi trường hệ điều hành Windows 10 (64-bit), bộ nhớ RAM 32 GB. Để đảm bảo tính công bằng, tất cả 4 thuật toán (BFS, DFS, UCS, A*) đều chạy trên cùng một engine luật chơi duy nhất và áp dụng chung một hàm chi phí.

Các chỉ số đo lường chính bao gồm:

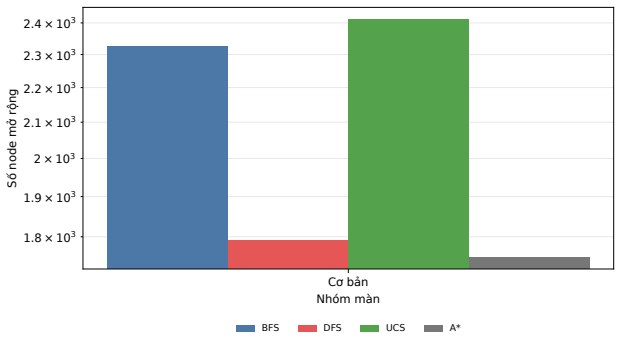
- **Thời gian tìm kiếm (ms):** Được đo lường thông qua thư viện `time`, lấy giá trị trung vị sau 3 lần chạy thử.
- **Bộ nhớ đỉnh (KB):** Được theo dõi qua thư viện `tracemalloc` trong một lượt chạy độc lập để không làm ảnh hưởng đến thời gian tính toán.
- **Số nút mở rộng:** Đếm số lượng trạng thái được lấy ra khỏi tập biên (frontier) để sinh hậu duệ.
- **Độ dài và chi phí (Cost):** Lời giải được đánh giá qua số bước đi thực tế và tổng chi phí. Hàm chi phí có trọng số quy định thao tác đổi tư thế tốn 2 đơn vị, trong khi các thao tác di chuyển giữ nguyên tư thế chỉ tốn 1 đơn vị.

5.3 Kết quả thực nghiệm

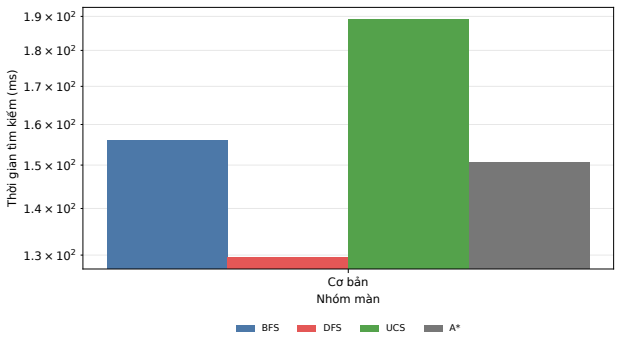
Bảng dưới đây trình bày kết quả trung bình của 4 thuật toán (BFS, DFS, UCS, A*) trên 22 màn chơi chuẩn, được phân chia theo từng cơ chế.

Nhóm màn	Thuật toán	Node mở rộng	Thời gian (ms)	Bộ nhớ (KB)	Độ dài	Cost
Cơ bản	BFS	91.3	1.70	29.3	12.00	21.33
	DFS	72.7	1.46	30.2	55.00	91.00
	UCS	94.7	1.99	36.9	12.00	21.33
	A*	77.0	1.76	38.4	12.00	21.33
Ô giòn	BFS	101.4	2.24	28.9	11.00	18.60
	DFS	91.6	2.09	40.0	80.40	130.40
	UCS	103.6	2.36	37.2	11.80	17.80
	A*	52.4	1.37	35.1	11.80	17.80
Công tắc	BFS	1490.2	35.21	673.7	16.25	25.88
	DFS	175.2	3.94	92.3	131.75	200.50
	UCS	1837.8	66.86	933.2	16.88	25.00
	A*	1412.6	46.62	824.4	17.38	25.00
Kết hợp	BFS	246.3	6.22	66.7	23.00	38.67
	DFS	205.8	5.79	86.7	131.33	208.33
	UCS	242.7	6.74	82.1	23.83	36.17
	A*	176.3	5.33	74.5	23.83	36.17

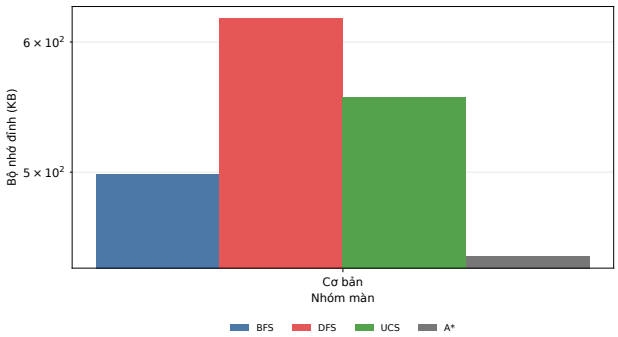
Bảng 3: Kết quả thực nghiệm trung bình của 4 thuật toán theo nhóm màn chơi.



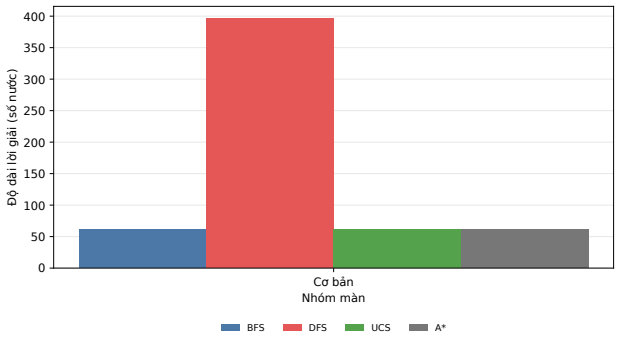
(a) Số node mở rộng trung bình (thang log)



(b) Thời gian tìm kiếm trung bình (ms)



(c) Bộ nhớ đỉnh trung bình (KB)



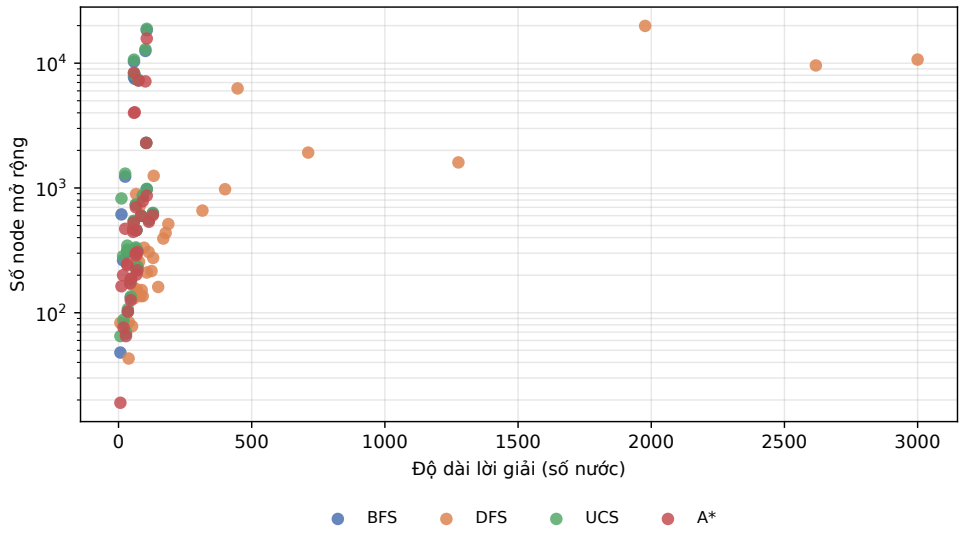
(d) Độ dài lời giải trung bình (số nước)

Hình 1: So sánh hiệu năng tổng thể giữa 4 thuật toán tìm kiếm.

5.4 Phân tích và đánh giá

Dựa trên các số liệu thực nghiệm và biểu đồ trực quan hóa, chúng tôi rút ra các đánh giá chi tiết cho từng chiến lược tìm kiếm:

- **Breadth-First Search (BFS):** Thuật toán luôn đảm bảo tìm ra số bước đi ngắn nhất. BFS hoạt động rất nhanh ở các màn cơ bản và ô giòn. Tuy nhiên, khi gặp cơ chế tách khối (split), không gian trạng thái phình to khiến số lượng nút mở rộng và bộ nhớ của BFS tăng vọt một cách đáng kể.
- **Depth-First Search (DFS):** Ưu điểm của DFS là thời gian chạy rất nhanh (vượt trội hơn BFS trên 16/22 màn) và ít bị ảnh hưởng bởi cơ chế tách khối. Tuy nhiên, chất lượng lời giải của DFS cực kỳ kém, độ dài đoạn đường trung bình dài gấp khoảng 6.9 lần so với mức tối ưu.
- **Uniform-Cost Search (UCS):** UCS luôn đảm bảo chi phí (cost) tối ưu nhất. Mặc dù UCS có thể đi nhiều bước hơn BFS, nhưng tổng chi phí luôn thấp hơn hoặc bằng. Hạn chế của UCS là tốc độ chậm hơn BFS do chi phí duy trì cấu trúc dữ liệu hàng đợi ưu tiên.
- **A* Search:** Bằng việc áp dụng heuristic khoảng cách Manhattan nới lỏng, thuật toán A* đạt được chi phí tối ưu trên 100% màn chơi giống hệt như UCS. Heuristic giúp A* tiết kiệm từ 18.7% đến 49.4% số lượng nút cần duyệt so với UCS, phát huy sức mạnh tốt nhất ở các màn chứa ô giòn. Dù vậy, thuật toán vẫn gặp đôi chút khó khăn ở màn tách khối do hàm heuristic đã nới lỏng chi phí điều phối hai nửa khối.



Hình 2: Tương quan giữa số node mở rộng và độ dài lời giải tối ưu.

6 Kết luận và hướng phát triển

6.1 Kết quả đạt được

Hệ thống đã triển khai thành công một Game Core Engine vững chắc với mô hình hóa trạng thái bất biến, đảm bảo tính chính xác và không xảy ra lệch tọa độ trong các phép lăn khối. Việc ứng dụng cấu trúc `frozenset` cho hệ thống cầu giúp các thao tác kiểm tra và chuyển trạng thái đạt độ phức tạp $O(1)$.

Bên cạnh đó, nhóm đã cài đặt và tích hợp hoàn chỉnh 4 thuật toán tìm kiếm không gian trạng thái (BFS, DFS, UCS, A*). Trong đó, thuật toán A* kết hợp với hàm Heuristic cải tiến đã chứng minh được tính tối ưu (Admissible và Consistent), giúp giảm đáng kể số lượng nút cần mở rộng so với tìm kiếm mù.

6.2 Tóm tắt hướng dẫn lựa chọn thuật toán

Từ các kết quả phân tích thực nghiệm trên bộ test chuẩn, nhóm rút ra định hướng lựa chọn thuật toán tối ưu cho bài toán Bloxorz như sau:

- **Lựa chọn A*:** Khi bài toán yêu cầu tìm ra đường đi có tổng chi phí (cost) tối ưu nhất, đặc biệt phát huy hiệu quả cao trên các bản đồ chứa ô giòn.
- **Lựa chọn BFS:** Khi cần tối ưu hóa tuyệt đối số lượng bước đi trong các bản đồ không có cơ chế tách khối (split).
- **Lựa chọn DFS:** Khi ưu tiên tốc độ tìm ra một lời giải hợp lệ trong thời gian ngắn nhất và chấp nhận độ dài đường đi không tối ưu.

6.3 Hướng phát triển

- **Cải tiến Heuristic (H_3):** Khắc phục hạn chế của khoảng cách Manhattan khi gặp vực trống (Void) hoặc cầu đóng, thông qua việc tiền tính toán khoảng cách thực tế (BFS Walkable Distance) và phân rã mục tiêu phụ (Sub-goal Decomposition) hướng tới các công tắc trước khi về hố đích.

- **Tối ưu hóa Core Engine:** Áp dụng Bitmask và Zobrist Hashing để mã hóa cấu hình hệ thống cầu trên một số nguyên 64-bit nhằm tối ưu dung lượng RAM và tăng tốc độ so sánh trạng thái.
- **Cắt tỉa nhánh sớm (Static Deadlock Detection):** Tích hợp phân tích và phát hiện các vùng bế tắc tĩnh ngay từ giai đoạn nạp bản đồ để sớm loại bỏ các nhánh trạng thái không thể dẫn tới lời giải.

Tài liệu tham khảo

[1] Coolmath Games, "Bloxorz - Coolmath Games," `https://www.coolmathgames.com/0-bloxorz`.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[4] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.

[5] A. Mez, "Bloxorz Solver Repository," GitHub, `https://github.com/AchrafMez/bloxorz`.

[6] Python Software Foundation, "`heapq` – Heap queue algorithm," Python 3 Documentation, `https://docs.python.org/3/library/heapq.html`.

[7] Python Software Foundation, "`tracemalloc` – Trace memory allocations," Python 3 Documentation, `https://docs.python.org/3/library/tracemalloc.html`.

[8] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

Phụ lục

Các liên kết

- **GitHub Repository:** `https://github.com/ng-h-nam/bloxorz-solver`
- **Video Demo:** `https://youtu.be/DkghNS2HQl8`

Kết quả chi tiết từng màn

Số node mở rộng

Màn	Nhóm	BFS	DFS	UCS	A*
Stage 01	Cơ bản	48	83	65	19
Stage 02	Cơ bản	263	283	284	200
Stage 03	Cơ bản	81	43	88	76
Stage 04	Cơ bản	68	84	70	65
Stage 05	Cơ bản	308	136	320	247
Stage 06	Cơ bản	102	78	107	102
Stage 07	Cơ bản	173	154	178	172
Stage 08	Cơ bản	614	161	824	163
Stage 09	Cơ bản	1232	1921	1303	471
Stage 10	Cơ bản	10292	1602	10671	8389
Stage 11	Cơ bản	186	169	187	187
Stage 12	Cơ bản	305	307	321	288
Stage 13	Cơ bản	132	130	135	126
Stage 14	Cơ bản	458	210	460	459
Stage 15	Cơ bản	7491	977	8003	4041
Stage 16	Cơ bản	308	894	345	240
Stage 17	Cơ bản	977	659	984	868
Stage 18	Cơ bản	598	393	599	598
Stage 19	Cơ bản	222	152	224	202
Stage 20	Cơ bản	7752	6274	8297	4001
Stage 21	Cơ bản	232	144	235	219
Stage 22	Cơ bản	325	216	334	300
Stage 23	Cơ bản	7298	19891	7322	7258
Stage 24	Cơ bản	539	136	545	528
Stage 25	Cơ bản	464	332	475	445
Stage 26	Cơ bản	18391	9589	18856	15766
Stage 27	Cơ bản	312	256	315	307
Stage 28	Cơ bản	12533	10686	12943	7143
Stage 29	Cơ bản	2296	1251	2299	2295
Stage 30	Cơ bản	552	275	557	536
Stage 31	Cơ bản	854	514	873	783
Stage 32	Cơ bản	626	436	631	605
Stage 33	Cơ bản	732	713	738	700

Thời gian tìm kiếm (ms)

Màn	Nhóm	BFS	DFS	UCS	A*
Stage 01	Cơ bản	2.67	3.61	3.71	1.73
Stage 02	Cơ bản	10.87	11.77	14.54	11.98
Stage 03	Cơ bản	3.20	2.41	3.62	3.71
Stage 04	Cơ bản	2.66	3.36	3.49	3.79
Stage 05	Cơ bản	11.75	5.98	13.74	13.15
Stage 06	Cơ bản	4.47	3.16	4.37	5.44
Stage 07	Cơ bản	6.57	5.80	7.43	8.14
Stage 08	Cơ bản	40.72	9.05	57.76	15.04
Stage 09	Cơ bản	65.76	110.43	81.02	34.06
Stage 10	Cơ bản	574.36	91.26	701.89	618.45
Stage 11	Cơ bản	7.12	6.47	7.38	8.71
Stage 12	Cơ bản	12.64	11.62	13.55	14.06
Stage 13	Cơ bản	5.48	4.83	5.15	6.11
Stage 14	Cơ bản	16.43	7.68	18.87	22.66
Stage 15	Cơ bản	430.03	47.46	554.87	287.75
Stage 16	Cơ bản	13.46	40.83	16.91	13.06
Stage 17	Cơ bản	37.76	25.74	43.52	47.33
Stage 18	Cơ bản	23.88	16.23	26.50	31.36
Stage 19	Cơ bản	7.80	6.10	8.79	8.90
Stage 20	Cơ bản	439.23	370.43	540.74	295.31
Stage 21	Cơ bản	8.61	5.02	9.20	10.70
Stage 22	Cơ bản	17.35	8.62	14.84	15.01
Stage 23	Cơ bản	398.96	1368.04	511.09	566.29
Stage 24	Cơ bản	20.75	6.02	23.40	26.21
Stage 25	Cơ bản	17.34	12.09	20.29	21.70
Stage 26	Cơ bản	1105.69	567.88	1351.60	1234.99
Stage 27	Cơ bản	12.32	10.62	13.78	15.57
Stage 28	Cơ bản	899.42	636.90	949.37	559.33
Stage 29	Cơ bản	107.99	51.60	112.93	123.31
Stage 30	Cơ bản	20.51	10.39	23.27	25.55
Stage 31	Cơ bản	32.67	21.12	38.35	39.89
Stage 32	Cơ bản	23.31	15.83	25.99	30.01
Stage 33	Cơ bản	32.66	31.37	40.23	41.44

Bộ nhớ đỉnh (KB)

Màn	Nhóm	BFS	DFS	UCS	A*
Stage 01	Cơ bản	21.1	26.6	21.1	10.1
Stage 02	Cơ bản	54.6	94.6	68.9	59.9
Stage 03	Cơ bản	26.6	17.7	23.6	16.7
Stage 04	Cơ bản	16.7	28.6	15.9	15.4
Stage 05	Cơ bản	96.6	49.7	76.3	67.9
Stage 06	Cơ bản	29.5	27.0	26.0	25.5
Stage 07	Cơ bản	38.6	41.1	41.3	41.4
Stage 08	Cơ bản	202.5	118.8	247.3	141.8
Stage 09	Cơ bản	345.8	804.7	333.9	230.3
Stage 10	Cơ bản	2112.8	971.5	2789.2	1923.9
Stage 11	Cơ bản	41.5	37.2	46.5	47.9
Stage 12	Cơ bản	89.2	94.7	71.1	63.6
Stage 13	Cơ bản	33.6	37.1	29.3	28.9
Stage 14	Cơ bản	98.7	55.0	92.1	93.7
Stage 15	Cơ bản	1852.0	427.2	1954.4	1056.8
Stage 16	Cơ bản	89.8	174.4	88.6	55.7
Stage 17	Cơ bản	179.3	211.8	201.9	203.7
Stage 18	Cơ bản	131.5	131.6	127.6	128.5
Stage 19	Cơ bản	56.2	46.4	58.7	55.1
Stage 20	Cơ bản	1891.2	2000.8	2002.0	1286.2
Stage 21	Cơ bản	50.7	37.8	53.8	52.5
Stage 22	Cơ bản	88.6	62.6	86.1	67.2
Stage 23	Cơ bản	1491.1	6864.5	1463.1	1592.7
Stage 24	Cơ bản	112.0	42.8	106.9	106.1
Stage 25	Cơ bản	111.3	98.0	107.4	103.7
Stage 26	Cơ bản	3354.8	3329.2	3968.0	3915.4
Stage 27	Cơ bản	90.7	53.5	67.0	65.0
Stage 28	Cơ bản	2697.2	3754.0	3085.4	2141.0
Stage 29	Cơ bản	450.7	340.8	449.2	450.4
Stage 30	Cơ bản	125.1	67.0	119.3	117.1
Stage 31	Cơ bản	176.0	145.0	198.1	185.1
Stage 32	Cơ bản	136.1	118.6	134.6	129.9
Stage 33	Cơ bản	157.0	154.4	183.2	187.5

Độ dài lời giải (số nước)

Màn	Nhóm	BFS	DFS	UCS	A*
Stage 01	Cơ bản	7	7	7	7
Stage 02	Cơ bản	17	68	17	17
Stage 03	Cơ bản	19	38	19	19
Stage 04	Cơ bản	28	39	28	28
Stage 05	Cơ bản	33	91	33	33
Stage 06	Cơ bản	35	51	35	35
Stage 07	Cơ bản	44	69	44	44
Stage 08	Cơ bản	11	149	11	11
Stage 09	Cơ bản	25	712	25	25
Stage 10	Cơ bản	58	1276	58	58
Stage 11	Cơ bản	47	49	47	47
Stage 12	Cơ bản	65	113	65	65
Stage 13	Cơ bản	46	57	46	46
Stage 14	Cơ bản	67	106	67	67
Stage 15	Cơ bản	61	400	61	61
Stage 16	Cơ bản	33	66	33	33
Stage 17	Cơ bản	106	315	106	106
Stage 18	Cơ bản	85	168	85	85
Stage 19	Cơ bản	67	87	67	67
Stage 20	Cơ bản	58	447	58	58
Stage 21	Cơ bản	71	72	71	71
Stage 22	Cơ bản	65	124	65	65
Stage 23	Cơ bản	75	1977	75	75
Stage 24	Cơ bản	57	83	57	57
Stage 25	Cơ bản	55	96	55	55
Stage 26	Cơ bản	106	2618	106	106
Stage 27	Cơ bản	71	77	71	71
Stage 28	Cơ bản	101	3000	101	101
Stage 29	Cơ bản	104	132	104	104
Stage 30	Cơ bản	114	130	114	114
Stage 31	Cơ bản	91	187	91	91
Stage 32	Cơ bản	129	177	129	129
Stage 33	Cơ bản	65	79	65	65

Cost

Màn	Nhóm	BFS	DFS	UCS	A*
Stage 01	Cơ bản	7.00	7.00	7.00	7.00
Stage 02	Cơ bản	17.00	68.00	17.00	17.00
Stage 03	Cơ bản	19.00	38.00	19.00	19.00
Stage 04	Cơ bản	28.00	39.00	28.00	28.00
Stage 05	Cơ bản	33.00	91.00	33.00	33.00
Stage 06	Cơ bản	35.00	51.00	35.00	35.00
Stage 07	Cơ bản	44.00	69.00	44.00	44.00
Stage 08	Cơ bản	11.00	149.00	11.00	11.00
Stage 09	Cơ bản	25.00	712.00	25.00	25.00
Stage 10	Cơ bản	58.00	1276.00	58.00	58.00
Stage 11	Cơ bản	47.00	49.00	47.00	47.00
Stage 12	Cơ bản	65.00	113.00	65.00	65.00
Stage 13	Cơ bản	46.00	57.00	46.00	46.00
Stage 14	Cơ bản	67.00	106.00	67.00	67.00
Stage 15	Cơ bản	61.00	400.00	61.00	61.00
Stage 16	Cơ bản	33.00	66.00	33.00	33.00
Stage 17	Cơ bản	106.00	315.00	106.00	106.00
Stage 18	Cơ bản	85.00	168.00	85.00	85.00
Stage 19	Cơ bản	67.00	87.00	67.00	67.00
Stage 20	Cơ bản	58.00	447.00	58.00	58.00
Stage 21	Cơ bản	71.00	72.00	71.00	71.00
Stage 22	Cơ bản	65.00	124.00	65.00	65.00
Stage 23	Cơ bản	75.00	1977.00	75.00	75.00
Stage 24	Cơ bản	57.00	83.00	57.00	57.00
Stage 25	Cơ bản	55.00	96.00	55.00	55.00
Stage 26	Cơ bản	106.00	2618.00	106.00	106.00
Stage 27	Cơ bản	71.00	77.00	71.00	71.00
Stage 28	Cơ bản	101.00	3000.00	101.00	101.00
Stage 29	Cơ bản	104.00	132.00	104.00	104.00
Stage 30	Cơ bản	114.00	130.00	114.00	114.00
Stage 31	Cơ bản	91.00	187.00	91.00	91.00
Stage 32	Cơ bản	129.00	177.00	129.00	129.00
Stage 33	Cơ bản	65.00	79.00	65.00	65.00

6.4 Sử dụng Generative AI

Bảng 4: Bảng tổng hợp cách sử dụng công cụ Generative AI của các thành viên

Thành viên	Công cụ	Dùng cho việc	Prompt tiêu biểu	Cách kiểm chứng
Hoàng Trung Phong	Claude AI	Hỗ trợ xây dựng Core Engine, logic chuyển trạng thái và xử lý đa luồng.	Kiểm tra logic kích hoạt cầu và tách khối để tránh lỗi rơi khối khi đóng cầu; gợi ý cách chạy solver dưới nền.	Viết unit test kiểm tra các phép lăn, hậu kiểm tra cầu sập và chạy thử trực tiếp với GUI.
Hà Chí Tâm	Gemini, Github Copilot	Tìm cách tối ưu thuật toán BFS, DFS, UCS.	Tìm hiểu về cơ chế cắt tỉa trong UCS, đối chiếu số liệu kết quả giữa các thuật toán để tìm phương pháp tối ưu nhất. Tìm hiểu cách chứng minh vì sao phương pháp đó là tối ưu nhất trong trường hợp này.	Chạy giải thử toàn bộ map, kiểm tra bộ nhớ tiêu thụ và đối chiếu lời giải giữa DFS, BFS và UCS.
Huỳnh Văn Phú	Claude AI, Gemini, Copilot	Thiết kế hàm Heuristic nâng cao, cài đặt A* và chứng minh tính chất toán học.	Kiểm tra và viết chứng minh tính admissible, consistent cho hàm heuristic khi khối bị phạt tư thế, điểm phạt chi tiết, tách khối.	Chạy bộ test tự động duyệt toàn bộ trạng thái trên các map nhỏ để đảm bảo không vi phạm admissibility.
Nguyễn Như Thuật	Claude AI	Phát triển giao diện 3D có tham khảo tựa game Minecraft, xử lí các hiệu ứng animation.	Hãy đề xuất cách chỉnh sửa để gạch dày hơn, nhìn góc nghiêng đẹp hơn. Hiệu ứng rơi khi khối nằm ngang chỉ đỡ trên một ô gạch chưa thật: đề xuất cách chỉnh.	Nhờ AI đề xuất trước hướng giải quyết vấn đề, được duyệt qua mới được áp dụng và chạy chương trình thực tế, thao tác thử các phím tắt, chọn màn, nhập mật khẩu và xem lại replay.
Nguyễn Hoàng Nam	Claude AI, Gemini	Viết script tự động chạy thực nghiệm, vẽ biểu đồ và rà soát tổng thể báo cáo LaTeX.	Viết script chạy thực nghiệm xuất biểu đồ; rà soát lỗi chính tả báo cáo.	Kiểm tra file dữ liệu xuất ra, đối chiếu biểu đồ với bảng số liệu trong báo cáo và biên dịch thử file PDF.